# Technical Report
# Integration of a Three-Position Rotating Rail Switch in the MFJA Third-Floor Gazebo Harmonic Workspace

MFJA Project Report

March 28, 2026

## Contents


**1 Introduction** **3**

**2 Problem Statement and Design Objectives** **3**
2.1 Functional requirement . . . 3
2.2 Why the moving part was not left as a purely physical joint . . . 4
2.3 Final design principles . . . 4

**3 Files and Their Roles** **4**

**4 Gazebo Model of the Moving Blade** **5**
4.1 Model intent . . . 5
4.2 Model structure in SDF . . . 5
4.3 Key decisions in the SDF file . . . 6

**5 World Integration and Placement** **6**
5.1 Placement in the full-floor world . . . 6
5.2 Placement in the room-315-only world . . . 7
5.3 Why placement belongs to the world files . . . 7

**6 Motion Control Logic** **7**
6.1 Overview . . . 7
6.2 How the script works . . . 9
6.3 Discrete switch states . . . 9
6.4 Quaternion conversion . . . 9
6.5 Set-pose service request . . . 9
6.6 Retries and partitions . . . 9

**7 How the Switch Is Installed and Exposed** **10**

**8 Runtime Workflow** **10**
8.1 Build step . . . 10
8.2 Launching the full-floor world . . . 10
8.3 Switching to one of the three states . . . 10
8.4 Room-315-only mode . . . 11

**9 Important Things to Know** **11**
9.1 The STL pivot is critical . . . . . 11
9.2 Do not reintroduce local mesh offsets unless necessary . . . . . 11
9.3 The final system is intentionally kinematic . . . . . 11
9.4 Global position and local model structure are different concepts . . . . . 11
9.5 Gazebo partitions must match . . . . . 11
9.6 Old helper scripts are not part of the final runtime . . . . . 12
9.7 The final runtime path is short and understandable . . . . . 12

**10 Future Improvements** **12**

**11 Conclusion** **12**

# 1 Introduction

This report explains in detail the engineering work carried out to add a new moving rail-switch element to the MFJA third-floor Gazebo Harmonic simulation. The final objective was not just to display a new mesh inside the environment. The real requirement was to integrate a rotating switch blade into the simulation in a way that is understandable, maintainable, reproducible, and easy to operate during demonstrations and experiments.

The added component is a three-position rotating switch located in the room-315 industrial cell. In practice, the moving element is the switch blade, while the left static cell remains a separate fixed model already present in the environment. The switch blade must be able to change between three calibrated orientations. At the end of the implementation, the selected orientations are:

- position `0` mapped to yaw angle `2.6` rad;
- position `1` mapped to yaw angle `-1.59` rad;
- position `2` mapped to yaw angle `0.50666` rad.

The final implementation deliberately uses a kinematic pose update strategy rather than a continuous physics-based joint controller. This choice is important. The requirement for the switch was to jump to one of three known states and remain there. It was not necessary to simulate the detailed dynamics of a mechanical actuator. During development, a purely physical solution proved harder to stabilize and more sensitive to pivot placement, inertia, damping, and collision behavior. For this reason, the final design directly sets the model pose in Gazebo using the `/world/<world_name>/set_pose` service.

This report describes:

- why the final design is kinematic instead of fully physical;
- which files were added or modified;
- where the moving part is defined as a Gazebo model;
- where the switch is inserted into the world files;
- which script is responsible for changing its orientation;
- how the motion command works internally;
- what practical details must be understood to maintain or extend the feature.

# 2 Problem Statement and Design Objectives

## 2.1 Functional requirement

The switch had to behave as a moving rail element inside the existing MFJA environment. More specifically, the implementation had to satisfy the following functional needs:

- integrate a moving switch blade mesh into the room-315 simulation;
- place the switch correctly inside both the full-floor world and the room-315-only world;
- allow the operator to command exactly three discrete switch states;
- preserve the correct pivot behavior, meaning the model should rotate around its intended local origin;
- keep the workflow simple for ROS 2 Jazzy and Gazebo Harmonic users.

### 2.2 Why the moving part was not left as a purely physical joint

An early idea was to build the switch with a revolute joint and a physics controller. From a modeling perspective, that is a reasonable first thought. However, in this specific case the physical approach created several practical problems:

- if the mesh pivot was not perfectly aligned, the blade rotated around a visually incorrect point;
- if the mass, inertia, collision settings, or damping were not tuned carefully, the blade could oscillate, overshoot, or keep moving;
- if the command interface used a physics controller topic, integration with ROS and Gazebo partitions became more fragile for a simple three-state component;
- the system requirement was discrete switching, not realistic actuation dynamics.

For these reasons, the final design favors a simpler and more robust behavior: the model is treated as a kinematic entity and its pose is updated directly to one of the calibrated states. This provides the exact user-facing behavior the project needed: a switch that goes immediately to the requested orientation and stays there.

### 2.3 Final design principles

The final solution follows four engineering principles:

1. **Keep the model origin meaningful.** The SolidWorks-exported STL is assumed to have the correct pivot and axis frame.
2. **Keep the world responsible for placement.** The global position of the switch belongs in the world files, not inside local mesh offsets.
3. **Keep the motion logic outside the model.** The movement command is handled by a dedicated Python tool that sends Gazebo pose requests.
4. **Keep the control states explicit.** The three allowed states are hard-coded as calibrated angles to avoid ambiguity.

## 3 Files and Their Roles

The switch integration is distributed across a small number of focused files. This separation is intentional, because it makes maintenance easier.

| File | Responsibility |
|---|---|
| `mfja_3rd_floor_description/models/rail_switch_3pos_left/model.config` | Declares the Gazebo model metadata and identifies the switch blade model. |
| `mfja_3rd_floor_description/models/rail_switch_3pos_left/model.sdf` | Defines the Gazebo representation of the moving blade, including its mesh, scale, and visual material. |
| `mfja_3rd_floor_description/models/rail_switch_3pos_left/meshes/aiguillage3.stl` | Stores the moving geometry exported from SolidWorks with the intended pivot and axes. |
| `mfja_3rd_floor_description/worlds/mfja_3rd_floor.world` | Places the moving switch inside the full third-floor world. |
| `mfja_3rd_floor_description/worlds/room_315_only.world` | Places the same switch inside the room-315-only world. |
| `mfja_robot_control_config/scripts/three_position_switch_demo.py` | Sends the discrete orientation command to Gazebo using the set-pose service. |
| `mfja_robot_control_config/CMakeLists.txt` | Installs the control script as a runnable ROS 2 utility. |

This organization is one of the most important things to understand. The moving blade is not controlled by embedding a behavior plugin inside the model. Instead, the project separates *description* from *command*. The SDF file describes what the blade looks like, while the Python script is responsible for changing its state.

# 4 Gazebo Model of the Moving Blade

## 4.1 Model intent

The final Gazebo model is intentionally minimal. It contains only the rotating blade, not the static rail base. The static base is already represented by a separate model in the world. This decision avoids duplicating geometry and keeps the moving object conceptually clear.

The corresponding model metadata is defined in `model.config`. It identifies the model under the name `rail_switch_3pos_left` and describes it as a three-position rotating rail switch that uses `aiguillage3.stl` as the moving element.

```
<?xml version="1.0"?>
<model>
  <name>rail_switch_3pos_left</name>
  <version>1.0</version>
  <sdf version="1.9">model.sdf</sdf>
  <author>
    <name>MFJA Project Team</name>
  </author>
  <description>Three-position rotating rail switch using the left static cell as base and aiguillage3.stl as the rotating element.</description>
</model>
```

Listing 1: Gazebo model metadata for the three-position switch blade.

## 4.2 Model structure in SDF

The actual Gazebo model is defined in the SDF file shown below.

```
<?xml version="1.0"?>
<sdf version="1.9">
  <model name="rail_switch_3pos_left">
    <!--
      This model is intentionally kinematic-only.
      The model origin is the desired switch pivot in world coordinates.
      We instantaneously change the whole model pose with /world/<name>/set_pose.
      The SolidWorks-exported mesh has the correct pivot / axes, so no local
      pose offset is needed here. Only a uniform mesh scale is applied.
    -->
    <link name="blade_link">
      <gravity>false</gravity>
      <inertial>
        <mass>0.1</mass>
        <inertia>
          <ixx>0.001</ixx>
          <ixy>0.0</ixy>
          <ixz>0.0</ixz>
          <iyy>0.001</iyy>
          <iyz>0.0</iyz>
          <izz>0.001</izz>
        </inertia>
      </inertial>

      <visual name="blade_visual">
        <geometry>
          <mesh>
            <uri>model://rail_switch_3pos_left/meshes/aiguillage3.stl</uri>
            <scale>39.37 39.37 39.37</scale>
          </mesh>
        </geometry>
        <material>
          <ambient>1.0 0.05 0.05 1</ambient>
          <diffuse>1.0 0.05 0.05 1</diffuse>
          <specular>0.45 0.12 0.12 1</specular>
          <emissive>0.08 0.0 0.0 1</emissive>
        </material>
      </visual>
    </link>
  </model>
</sdf>
```

Listing 2: Final SDF model used for the moving switch blade.

### 4.3 Key decisions in the SDF file

Several implementation choices in this file are especially important:

- The model contains only one link, `blade_link`.
- Gravity is disabled for the link, because the blade is not meant to be simulated as a falling rigid body.
- A very small inertial block is kept for Gazebo compatibility, but the switch is not meant to behave like a physics-driven object.
- The mesh URI points to `aiguillage3.stl`, the corrected SolidWorks export.
- No local pose offset is applied inside the model. This is a major design statement: the mesh itself is expected to carry the correct pivot and axis frame.
- A uniform scale is applied to match the scene because the SolidWorks export is smaller than the legacy in-scene geometry.

This last point deserves emphasis. If the STL had a wrong pivot, the implementation would have required local offsets or a different link structure. Because the final workflow assumes the corrected SolidWorks mesh is already aligned, the SDF can remain clean.

## 5 World Integration and Placement

### 5.1 Placement in the full-floor world

The full-floor world includes the left static cell and then the moving blade model. The following snippet shows the insertion of the switch into the main world file.

```
<include>
  <uri>model://cell_static_gauche_final</uri>
  <name>cell_static_gauche_final_1</name>
  <pose>-16.9 -5.81 0.0 0.0 0.0 0.0</pose>
</include>

<include>
  <uri>model://rail_switch_3pos_left</uri>
  <name>rail_switch_3pos_left</name>
  <pose>-16.73 -4.42 0.73 0.0 0.0 -1.59</pose>
</include>
```

Listing 3: Switch insertion in the full-floor Gazebo world.

The calibrated pose used for the moving switch is:

- `x = -16.73`
- `y = -4.42`
- `z = 0.73`
- initial `yaw = -1.59`

In the final design, this global pose is the correct place to adjust the switch location in the environment. If the blade must be moved slightly within the room, the world pose should be updated. The model SDF should generally remain unchanged as long as the exported mesh pivot is valid.

### 5.2 Placement in the room-315-only world

The same concept is reused in the room-315-only world. This is useful because the project supports both a full-floor mode and a focused local test mode. The switch appears in the same calibrated place relative to the room-315 geometry.

```
    <include>
      <uri>model://cell_static_gauche_final</uri>
      <name>room315_cell_static_gauche_final_1</name>
      <pose>-16.9 -5.81 0.0 0.0 0.0 0.0</pose>
    </include>

    <include>
      <uri>model://rail_switch_3pos_left</uri>
      <name>rail_switch_3pos_left</name>
      <pose>-16.73 -4.42 0.73 0.0 0.0 -1.59</pose>
    </include>
```

Listing 4: Switch insertion in the room-315-only Gazebo world.

### 5.3 Why placement belongs to the world files

One of the most important lessons from this work is that local offsets inside the model and global placement in the world must not be mixed carelessly. The final arrangement is easier to reason about because:

- `model.sdf` answers the question: "What is this moving object?"
- the world file answers the question: "Where is this object in the environment?"
- the control script answers the question: "Which calibrated orientation should this object take now?"

Keeping these responsibilities separate makes debugging much easier.

## 6 Motion Control Logic

### 6.1 Overview

The runtime motion logic is implemented in the script `three_position_switch_demo.py`. This script is the main control entry point for the switch. It is not just a demo in the informal sense. In the final design, it is the utility that encodes the three allowed switch states and sends the corresponding pose command to Gazebo.

```
#!/usr/bin/env python3

import argparse
import math
import os
import subprocess
import sys
import time


SWITCH_POSITIONS = {
    '0': 2.6,
    '1': -1.59,
    '2': 0.50666,
}

def quaternion_from_yaw(yaw: float):
    half = yaw * 0.5
    return 0.0, 0.0, math.sin(half), math.cos(half)


def main():
    parser = argparse.ArgumentParser(
        description='Instantly set the three-position rail switch to one of the calibrated yaw angles.'
    )
    parser.add_argument(
        'position',
        choices=sorted(SWITCH_POSITIONS),
        help='Discrete switch position: 0, 1, or 2',
    )
    parser.add_argument(
        '--world',
        default='default',
```

```
        help='Gazebo world name. Use "default" for the full floor and "room_315_only" for the room-only world.',
    )
    parser.add_argument(
        '--entity',
        default='rail_switch_3pos_left',
        help='Gazebo model name of the moving switch blade.',
    )
    parser.add_argument(
        '--pos',
        nargs=3,
        type=float,
        default=[-16.73, -4.42, 0.73],
        metavar=('X', 'Y', 'Z'),
        help='World position of the switch pivot model origin.',
    )
    parser.add_argument(
        '--timeout-ms',
        type=int,
        default=1500,
        help='Timeout passed to gz service.',
    )
    parser.add_argument(
        '--retries',
        type=int,
        default=3,
        help='How many times to retry the gz set_pose service before failing.',
    )
    parser.add_argument(
        '--partition',
        default=os.environ.get('GZ_PARTITION', 'rail_switch_demo'),
        help='Gazebo transport partition. Defaults to $GZ_PARTITION or rail_switch_demo.',
    )
    args = parser.parse_args()

    yaw = SWITCH_POSITIONS[args.position]
    qx, qy, qz, qw = quaternion_from_yaw(yaw)
    x, y, z = args.pos

    req = (
        f'name: "{args.entity}", '
        f'position: {{x: {x}, y: {y}, z: {z}}}, '
        f'orientation: {{x: {qx}, y: {qy}, z: {qz}, w: {qw}}}'
    )
    cmd = [
        'gz', 'service',
        '-s', f'/world/{args.world}/set_pose',
        '--reqtype', 'gz.msgs.Pose',
        '--reptype', 'gz.msgs.Boolean',
        '--timeout', str(args.timeout_ms),
        '--req', req,
    ]

    print(
        f'Setting {args.entity} in world {args.world} to position {args.position} '
        f'({yaw:.3f} rad) at pose ({x}, {y}, {z}) on partition {args.partition}.'
    )
    env = os.environ.copy()
    env['GZ_PARTITION'] = args.partition
    last_output = ''
    for attempt in range(1, args.retries + 1):
        completed = subprocess.run(
            cmd,
            check=False,
            env=env,
            text=True,
            capture_output=True,
        )
        output = '\n'.join(part for part in [completed.stdout, completed.stderr] if part).strip()
        last_output = output
        if output:
            print(output)

        timed_out = 'timed out' in output.lower()
        success = completed.returncode == 0 and not timed_out
        if success:
            print(f'Done on attempt {attempt}.')
            return

        if attempt < args.retries:
            print(f'Attempt {attempt} failed, retrying...')
            time.sleep(0.25)

    raise SystemExit(
        f'Failed to set switch pose in world {args.world} on partition {args.partition}. '
        f'Last output: {last_output}'
    )


if __name__ == '__main__':
    try:
        main()
    except subprocess.CalledProcessError as exc:
        print(f'Failed to set switch pose: {exc}', file=sys.stderr)
        raise SystemExit(exc.returncode)
```

Listing 5: Python control utility for the three-position switch.

### 6.2 How the script works

The script performs the following steps:

1. parse the requested switch position from the command line;
2. map the symbolic position number (`0`, `1`, or `2`) to a calibrated yaw angle;
3. convert that yaw value into a quaternion because Gazebo pose requests use quaternion orientation;
4. combine the chosen orientation with the calibrated world position of the switch pivot;
5. call `gz service` on `/world/<world_name>/set_pose`;
6. retry a few times if the service call times out.

### 6.3 Discrete switch states

The final angle mapping stored in the script is:

- `position 0 -> 2.6 rad`
- `position 1 -> -1.59 rad`
- `position 2 -> 0.50666 rad`

This mapping is intentionally explicit. The operator is not asked to send arbitrary angles. This protects the system from inconsistent use and matches the physical interpretation of a real three-position switch.

### 6.4 Quaternion conversion

Although the switch is conceptually driven by a yaw angle, Gazebo requires the orientation to be sent as a quaternion. The helper function `quaternion_from_yaw()` converts a scalar yaw angle into the `z,w` quaternion pair used for a planar rotation around the vertical axis. This keeps the actual command interface compatible with Gazebo while preserving a human-readable notion of switch states.

### 6.5 Set-pose service request

The command is sent through:

`/world/<world_name>/set_pose`

This is a core design decision. Instead of asking Gazebo to solve a moving joint over time, the code simply requests the final state directly. For a three-position industrial switch, this is often preferable during simulation development because it is deterministic, immediate, and easy to reproduce.

### 6.6 Retries and partitions

The script also includes two practical reliability features:

- **Retries:** if the service temporarily times out, the script tries again.
- **Gazebo partition support:** the script reads or sets `GZ_PARTITION`, which is necessary when the simulation is isolated inside a custom Gazebo transport partition.

This is an important operational detail. A correct script can still appear to fail if the terminal used to send the command is not using the same Gazebo partition as the running simulation.

# 7 How the Switch Is Installed and Exposed

The control utility becomes runnable through ROS 2 because it is installed by `mfja_robot_control_config/CMakeLists.txt`. The relevant part is shown below.

```
cmake_minimum_required(VERSION 3.28.0)
project(mfja_robot_control_config)

find_package(ament_cmake REQUIRED)

install(
  DIRECTORY config launch
  DESTINATION share/${PROJECT_NAME}
  PATTERN "__pycache__" EXCLUDE
  PATTERN "*.pyc" EXCLUDE
)

install(
  PROGRAMS
    scripts/multi_robot_sync_demo.py
    scripts/shift_stl_origin.py
    scripts/three_position_switch_demo.py
  DESTINATION lib/${PROJECT_NAME}
)

ament_package()
```

Listing 6: Installation of the switch utility as a ROS 2-executable Python script.

This file matters for practical reasons:

- without this installation rule, the package executable `three_position_switch_demo.py` could not be launched with `ros2 run`;
- it keeps the runtime interface consistent with the rest of the workspace tools;
- it makes the switch control part of the package's deployable API.

# 8 Runtime Workflow

## 8.1 Build step

After modifying the switch-related files, the relevant packages should be rebuilt:

```
cd ~/mfja_3rd_floor_ws
source /opt/ros/jazzy/setup.bash
colcon build --packages-select \
  mfja_3rd_floor_description \
  mfja_robot_control_config \
  mfja_3rd_floor_bringup \
  mfja_room_315_bringup \
  --symlink-install
source install/setup.bash
```

Listing 7: Typical rebuild command for the switch-related packages.

## 8.2 Launching the full-floor world

The switch is available in the main full-floor simulation:

```
export GZ_PARTITION=rail_switch_demo
ros2 launch mfja_3rd_floor_bringup full_floor.launch.py \
  robots:=none \
  gz_partition:=rail_switch_demo
```

Listing 8: Example launch command for the full-floor world.

## 8.3 Switching to one of the three states

The operator can then request one of the three calibrated positions:

```
ros2 run mfja_robot_control_config three_position_switch_demo.py 0 \
  --partition rail_switch_demo \
  --pos -16.73 -4.42 0.73

ros2 run mfja_robot_control_config three_position_switch_demo.py 1 \
  --partition rail_switch_demo \
  --pos -16.73 -4.42 0.73

ros2 run mfja_robot_control_config three_position_switch_demo.py 2 \
  --partition rail_switch_demo \
  --pos -16.73 -4.42 0.73
```

Listing 9: Example commands for the three discrete switch positions.

### 8.4 Room-315-only mode

The same approach works in the isolated room-315-only world:

```
export GZ_PARTITION=rail_switch_demo
ros2 launch mfja_room_315_bringup room_315_only.launch.py \
  robots:=none \
  gz_partition:=rail_switch_demo
```

Listing 10: Example command for the room-315-only switch test.

In that case, the script should be called with:

`-world room_315_only`

## 9 Important Things to Know

### 9.1 The STL pivot is critical

The most important modeling fact is that Gazebo rotates a model around its origin. If the STL origin is wrong, the switch will rotate around the wrong point even if the yaw commands are correct. The current solution assumes that `aiguillage3.stl` exported from SolidWorks has the right pivot and the right axis orientation.

### 9.2 Do not reintroduce local mesh offsets unless necessary

If the switch placement seems wrong in the room, the first place to check is the world pose. Reintroducing local offsets into `model.sdf` should be avoided unless the mesh itself is later changed again and requires compensation.

### 9.3 The final system is intentionally kinematic

The switch does not currently use a revolute joint, a PID controller, or a ROS-to-Gazebo command bridge. That is not a limitation of the current code; it is a deliberate simplification chosen to make a three-state device reliable.

### 9.4 Global position and local model structure are different concepts

The values `x y z` passed through `-pos` or defined in the world file represent the switch pivot in world coordinates. They are not mesh-internal offsets. This distinction is central to understanding the whole design.

### 9.5 Gazebo partitions must match

If a command appears not to work, one of the first things to verify is that the terminal sending the command uses the same `GZ_PARTITION` as the running simulation. A mismatch can make the control command appear to fail even if the script is correct.

### 9.6 Old helper scripts are not part of the final runtime

The workspace may still contain helper utilities such as `shift_stl_origin.py`. Such scripts are useful during mesh preparation, but they are not part of the final runtime motion pipeline. The runtime behavior depends mainly on:

- the corrected STL file;
- the world pose entries;
- the `three_position_switch_demo.py` control script.

### 9.7 The final runtime path is short and understandable

This is the simplest way to mentally model the final implementation:

1. Gazebo loads the blade mesh from `model.sdf`.
2. The world file places the blade at the calibrated switch pivot location.
3. The Python control script sends one of three allowed orientations.
4. Gazebo updates the blade pose immediately.

## 10 Future Improvements

Although the current solution is appropriate for the project requirements, several future improvements are possible:

- add a small ROS 2 node wrapper around the script so other nodes can request switch states programmatically;
- define semantic command names such as `left`, `center`, and `right` instead of numeric identifiers;
- log the currently requested switch state to a ROS topic for monitoring or user interfaces;
- create a dedicated launch file that opens a minimal world containing only the switch and its local environment for faster testing;
- if a more physical behavior is ever required, introduce a new model variant that uses a true revolute joint while keeping the current kinematic version for demonstrations.

## 11 Conclusion

The moving rail-switch element added to the MFJA third-floor project is more than a visual asset. It is now a properly integrated simulation feature with a clear architecture:

- the mesh geometry lives in the description package;
- the model is defined cleanly in Gazebo as a dedicated blade model;
- the world files place it consistently in both simulation modes;
- the Python tool controls its three discrete states in a practical and deterministic way.

The most important outcome of this work is conceptual clarity. The final solution works because it separates concerns:

- geometry belongs to the mesh;
- appearance belongs to the SDF model;
- placement belongs to the world file;
- state changes belong to the control script.

This separation makes the system easier to debug, explain, extend, and demonstrate. It also means the moving switch can now be treated as a reusable simulation component rather than a one-off mesh experiment.